JOURNAL OF
Optical Communications
and Networking

Real-time capable, low-latency upstream scheduling in multi-tenant, SLA compliant TWDM PON

ARIJEET GANGULI* AND MARCO RUFFINI

School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland
**gangulia@tcd.ie*

Received 28 January 2025; revised 20 April 2025; accepted 27 April 2025; published 23 May 2025

Virtualized passive optical networks (vPONs) offer a promising solution for modern access networks, bringing enhanced flexibility, reduced capital expenditures, and support for multi-tenancy. By decoupling network functions from the physical infrastructure, vPONs enable service providers to efficiently share network resources among multiple tenants. In this paper we propose a novel, to our knowledge, merging DBA algorithm called the dynamic time and wavelength allocation algorithm for a virtualized DBA architecture in multi-tenant PON environments. The algorithm, which enables the merging of multiple virtual DBAs into a physical bandwidth map, introduces multi-channel support, allowing each optical network unit (ONU) to dynamically change, taking into consideration different switching times and transmission wavelengths. Leveraging the Numba APIs for high-performance optimization, the algorithm achieves real-time performance with minimal additional latency, meeting the stringent requirements of SLA-compliant, latency-critical 6G applications and services. Our analysis highlights an important trade-off in terms of throughput under multi-tenant conditions, between single-channel versus multi-channel PONs, as a function of ONU tuning time. We also compare the performance of our algorithm for different traffic distributions. Finally, in order to assess the time computing penalty of dynamic wavelength optimization in the merging DBA algorithm, we compare it against a baseline static wavelength allocation algorithm, where ONUs are designated a fixed wavelength for transmission.

https://doi.org/10.1364/JOCN.558325

1. INTRODUCTION

As the development of 6G networks takes pace, the telecommunication industry faces numerous challenges to accommodate exponential data traffic, massive connectivity, and increasingly diverse quality of service (QoS) requirements. 6G builds on the foundation of 5G while pushing the boundaries of network capabilities to support ultra-high throughput, extremely low end-to-end latency (of the order of 1 ms) [1], and deterministic QoS for a new era of services and applications. Passive optical networks (PONs) have emerged as a critical technology in the evolution of these access networks, offering a cost-effective, scalable, reliable, and high-speed solution for delivering broadband services to a wide range of users [2]. With the demand for higher bandwidth and lower latency, particularly in the context of 5G and beyond, PONs are playing an increasingly vital role in both residential and enterprise applications. Their ability to deliver symmetric data rates, such as in the multi-channel 40G and single-channel 50G-PON [3,4], makes them a suitable candidate for supporting next-generation applications, including 6G fronthaul and other latency-sensitive services.

Traditional PON architectures are somewhat rigid, with a single network operator controlling the optical line terminal (OLT) and managing bandwidth allocation through a centralized dynamic bandwidth allocation (DBA) mechanism. But over the past decade, network architectures have undergone a significant transition from closed systems to open and disaggregated systems. Software defined networking (SDN) and network function virtualization (NFV) have played a key role, offering programmability, cost efficiency, and flexibility. PONs have experienced a similar transition toward virtualization [5,6], which enables dynamic software-based control of scheduling algorithms and multi-tenancy. This allows different virtual network operators (VNOs) to run multiple independent upstream scheduling algorithms in a shared PON. This maximizes the usage of available bandwidth while significantly reducing operational costs through sharing of network

1943-0620/25/060485-13 Journal © 2025 Optica Publishing Group

resources, making it a highly scalable and flexible solution for next-generation network services [7].

Due to their cost-effectiveness and high scalability, virtualized passive optical networks (vPONs) can be used to support mobile fronthaul technology for the Open Radio Access Network (O-RAN) [8]. This can be achieved by connecting small cells in a 5G functional split architecture (i.e., supporting an O-RAN 7.2 remote unit (RU)–distributed unit (DU) split as well as higher level splits) using a PON point-to-multipoint (P2MP) topology. This can help reduce the capital expenditure (CAPEX) and operational expenditure (OPEX) for the fronthaul network compared to the traditional point-to-point (P2P) topology. However, P2MP fronthaul introduces additional latency within the PON system due to upstream scheduling, as typically the PON and DU upstream schedulers operate independently. Thus, in order to support applications that require Ultra-Reliable Low-Latency Communications (URLLC), the concept of cooperative DBA was originally introduced in [9]. This focuses on the development of an optical-mobile cooperative interface that translates mobile upstream scheduling data into PON transmit request information. The concept was later standardized as the Cooperative Transport Interface (CTI) [10], which facilitates enhanced coordination between PON and RAN upstream schedulers, leading to more efficient resource allocation and reduced latency.

A. Literature Review

Early DBA algorithms in traditional PON architectures focused primarily on fairness and efficiency. The most common approach has been the grant-request model, where each optical network unit (ONU) sends a bandwidth request to the OLT, which then allocates the available bandwidth. The classical interleaved polling with adaptive cycle time (IPACT) algorithm, introduced in [11], is one of the foundational DBA schemes used in PONs. IPACT dynamically adjusts polling cycles based on traffic conditions, reducing idle time and improving bandwidth utilization. However, while IPACT provides an efficient framework for bandwidth allocation, it faces challenges in scenarios with diverse traffic patterns, especially when serving multiple services with distinct QoS requirements.

Subsequent DBA algorithms introduced hierarchical bandwidth allocation schemes, based on traffic priorities. One such network traffic prioritization algorithm is presented in [12], where optimized versions of the GigaPON Access Network (GIANT) DBA called deficit XGIANT (XGIANT-D) and proportional XGIANT (XGIANT-P) were proposed. It was demonstrated that both XGIANT-D and XGIANT-P ensure strictly prioritized, low, and fair mean queuing delays for aggregated voice, video, and best-effort upstream traffic under two loaded conditions on 10-Gbit capable PONs (XG-PONs). Moreover, XGIANT-D and XGIANT-P also ensure reduced packet losses for aggregated upstream traffic in the XG-PON-based LTE backhaul compared to GIANT and efficient bandwidth utilization algorithms. Another novel DBA algorithm supporting QoS and a services hierarchy is proposed in [13] and divides network services according to different priorities, resulting in improved channel utilization and reduced average delay.

As PONs evolved to accommodate multi-service networks, research work on DBA algorithms has started to incorporate service level agreement (SLA)-based bandwidth allocation mechanisms to ensure compliance with contracted QoS levels. SLA-aware DBA algorithms consider the bandwidth requests from ONUs and prioritize traffic based on latency and bandwidth guarantees, ensuring higher-priority traffic, such as video streaming or 5G fronthaul, receives timely allocation. For instance, the delay-aware DBA (DA-DBA) algorithm [14] enhances traditional DBA schemes by factoring in the delay sensitivity of different traffic types. The DA-DBA approach dynamically adjusts bandwidth allocations based on the delay profiles of active flows, ensuring that latency-sensitive traffic is prioritized while still optimizing overall network efficiency. Another important development is the predictive DBA, which leverages traffic forecasting to allocate bandwidth in advance based on predicted demand [15].

The next evolutionary step was enabling multi-tenancy by allowing multiple service providers to share the infrastructure. True multi-tenancy necessitates full virtualization of the PON, allowing VNOs to control the PON protocol and perform capacity scheduling as if they owned and controlled the physical OLT devices [16]. In this scenario, service providers have more control over their DBAs, which are designed and customized to support a broader range of service agreements, such as low latency. In this regard, the authors of [17] introduced an inter-frame bandwidth resource-sharing framework for XG-PONs, where a fixed time slot frame of 125 μs is considered a slice. An operator is assigned an entire slice based on the proposed algorithm. This proposed framework allows for greater customizability, network isolation, and efficient bandwidth utilization. Nonetheless, it is observed that this solution is susceptible to greater delays, since bandwidth is likely to be wasted due to underutilized frames by specific operators. The authors extend this work in [18] to schedule whole frames to individual VNOs in a round-robin fashion. This approach, however, lacks the ability to mix multi-vendor allocations within each frame, resulting in latency that is dependent on the number of VNOs sharing the infrastructure. In [19], the authors present a novel dynamic merging algorithm that merges multiple bwmaps based on traffic priority such as assured, best effort, unassured etc. In [20], the authors present a novel load-adaptive merging algorithm (LAMA) that modifies the existing strict priority scheme called the priority-based merging algorithm (PBMA) scheme and improves the performance of the merging engine by allocating the physical bandwidth map in a load adaptive manner to the multi-tenant PON architecture.

All the work mentioned above considers a strict time-invariant traffic prioritization approach while allocating DBA. Different packets are marked with different priority levels and then queue management algorithms are applied to determine how packet prioritization is applied. While these can be effective in assigning relative importance to different services, they are unable to capture the complexity of specific SLAs, especially in a heterogeneous and shared network environment. This shortcoming leads to inefficient utilization of resources and the inability to guarantee performance. In [21], the authors presented a stateful SLA-compliant PON hypervisor that runs a merging DBA algorithm with a stateful

mechanism where the bandwidth allocations are made minimizing the breach of SLAs over time. In [22], we present a real-time stateful SLA-compliant merging algorithm capable of running multiple SLA-compliant services and applications. In [23], we present a multi-tenant multi-wavelength upstream transmission scheme for virtualized PONs, enabling compliance with latency-oriented SLAs called the dynamic time and wavelength allocation (DTWA) algorithm. The study demonstrated an important trade-off between the latency of multi-channel systems and the transmitters' tuning time. If the tuning time is negligible compared to the burst overhead, the multi-channel system has better latency performance. This is due to the fact that the burst overhead tends to have a similar time duration independently of the line rate [4]; thus higher line rate channels require a proportionally larger amount of bits than lower line rate channels. But as the ONU tuning time increases, the single-channel system outperforms the multi-channel ones. However, through network traffic simulations, we observed that the dynamic wavelength and time slot allocation algorithm is computationally intensive and added notable additional latency to the PON upstream scheduling mechanism.

B. Contribution

The work presented here is an extension of our previous work [23]. Here, we first extend the virtual DBA (vDBA) architecture for a multi-channel PON, where the merging of virtual bandwidth maps (vBmaps) is carried out across multiple wavelength channels. The key objective is to define a number of SLAs, expressed in terms of maximum latency and compliance level, and to propose an algorithm (real-time DTWA) that can operate the bandwidth map merging operation, minimizing the probability of breaching SLAs, across all VNOs. The evaluation of our work is based on a few key performance analyses. First, we compare the latency for systems using different line rates and numbers of channels. Assuming an overall PON capacity of 200 Gb/s, we consider a system with eight channels at 25 Gb/s, one with four channels running at 50 Gb/s, and one with a single channel at 200 Gb/s (being a single channel, there is no multi-wavelength allocation for the 200 Gb/s option). Then we show how the difference in performance changes when we consider different tuning times for the ONU transmitters. For this we compare the case of negligible tuning time (i.e., if the system implements channel bonding [24], so that the ONU has more than one transceiver always ready to transmit at least at another wavelength): the case of Class 1 transmitters (i.e., tuning time < 10 μs) [3], with a tuning time of 250 ns (i.e., close to the burst overhead time) and 1 μs, and the case of Class 2 transmitters (i.e., tuning time > 10 μs but < 25 ms), with a tuning time of 15 μs. Class 3 devices are not considered as they have tuning times longer than 25 ms, which is more than two orders of magnitude higher than the PON frame and would in practice represent a semi-static allocation.

Then, we extend the results by varying the network traffic distribution from the uniform traffic distribution considered in [23] to different distributions, namely, the Poisson, Zipf–Mendelbrot, and Pareto distributions. We show that the performance of our algorithm has some dependency on the traffic distribution. The performance is higher for uniformly distributed traffic (low variance) with higher expected arrival times. We also compare the DTWA algorithm with a static wavelength allocation (SWA) algorithm. On the one hand, we assess the improvement in latency that dynamic wavelength brings with respect to a static allocation. On the other hand, we also compare the merging engine algorithm execution time for the two algorithms, highlighting an important trade-off. This work also includes optimization of runtime execution by leveraging the just-in-time (JIT) compiler for Python, Numba, reporting the results for both static and dynamic wavelength allocation algorithms, showing yet another important trade-off between static and dynamic wavelength allocation algorithms.

2. SYSTEM ARCHITECTURE AND PROPOSED APPROACH

The virtual DBA architecture for a multi-channel system addressed in this work is shown in Fig. 1. In our approach, multiple VNOs run different schedulers in parallel (vDBAs), each forwarding a vBmap, which allocates upstream transmission channels and slots to a group of ONUs. The scheduling hypervisor (or merging engine) collects all such vBmaps that contain information about the requested bandwidth from different ONUs belonging to the respective VNOs for a specific interval of time. In this multi-wavelength approach, the OLT also has the option to select transmission over different channels, to minimize grant delay, although this is constrained by the ONU tuning time. The merging engine allocates a transmission channel and time slot within the channel for the allocations of all the Bmaps. Since each ONU is tuned to a specific wavelength at any point in time, the OLT can broadcast the allocated channel and time slot information downstream to all the ONUs using vBmaps through each of the transmission channels, and the ONUs that listen to that channel can access the vBmap information and can schedule

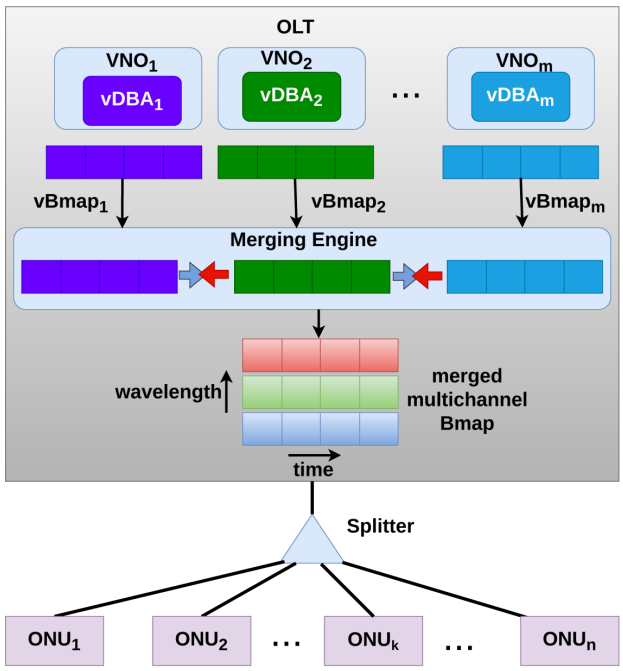

Fig. 1. vDBA architecture for the multi-tenant TWDM PON.

their transmission accordingly. As shown in Fig. 1, the TWDM system allows transmission in three different channels, and the merging engine broadcasts the allocated channel and slot information to the respective ONUs in the form of vBmaps through each of these channels. It should be noted that downstream and upstream channels can be selected independently for each ONU. Each bandwidth map will contain information on what time slot and upstream wavelength should be used by each ONU. It should also be noted that, in this work, we do not address how the downstream wavelength is selected, as this can be typically organized by the management layer and is not part of the upstream allocation problem. Our merging engine resolves allocation conflicts among vBmaps based on specific SLAs so that it minimizes the probability of breaching SLAs. The use of a stateful algorithm, which takes into consideration the history of a service flow when making scheduling prioritization decisions, is preferred to stateless algorithms [21]. This is because a stateful algorithm can prioritize flows depending on how close they are to breach their specific SLA target [22].

Thus, in this work, we propose a heuristic stateful TWDM scheduling algorithm. The objective of the algorithm is to maximize SLA compliance across all the flows during upstream transmission. We focus specifically on the additional latency introduced by the multi-tenancy sharing aspect of the PON. An SLA breach occurs when a given flow accumulates a number of delayed upstream slots that are above its target SLA threshold. For example, for an SLA with a maximum merging delay of 25 μs with 99% compliance, every time an upstream slot is delayed by more than 25 μs with respect to the requested time slot in the virtual Bmap, we increment a counter. If the counter goes above the non-compliance rate (in this case $100 - 99 = 1\%$), calculated over a number of frames (i.e., we use a 1 ms window, which is the time duration of a 5G sub-frame), then we consider that an SLA breach has occurred.

The problem of upstream wavelength and time slot allocation is formulated as a mixed integer programming (MIP) problem. Equation (1) calculates the maximum delay of any given allocation to remain within the target threshold for an SLA breach. We maintain a four-dimensional binary decision variable matrix $\boldsymbol{X}$, where the objective function [Eq. (2)] is explained as follows—the inner truth value function calculates the packet-level breach, and the outer truth value function calculates the flow-level breach across each virtual Bmap. Equation (3) is the constraint that any particular channel at any time slot transmits at most one Bmap allocation. Equation (4) is the constraint that all Bmap allocations are alloted unique channel–time slot pairs. Equation (5) checks the conservation of Bmap allocations within each virtual Bmap.

$$t_{i,j}^{max} = t_{i,j}^{req} + lat_i, \tag{1}$$

$$\min \sum_{i=1}^{N} \mathbb{I}\left(\frac{1}{|S_i|} \sum_{j=1}^{F} \sum_{k=1}^{W} \sum_{l=1}^{F} \mathbb{X}_{i,j,k,l} \mathbb{I}(l > t_{i,j}^{max}) > br_i^{pt} \right), \tag{2}$$

$$\text{s.t.} \quad \sum_{i=1}^{N} \sum_{j=1}^{F} \mathbb{X}_{i,j,k,l} \leq 1, \quad \forall\, 1 \leq k \leq W,\ 1 \leq l \leq F, \tag{3}$$

$$\sum_{k=1}^{W} \sum_{l=1}^{F} \mathbb{X}_{i,j,k,l} \doteq 1, \quad \forall\, 1 \leq i \leq N,\ 1 \leq j \leq F, \tag{4}$$

$$\sum_{j=1}^{F} \sum_{k=1}^{W} \sum_{l=1}^{F} \mathbb{X}_{i,j,k,l} \doteq |S_i|, \quad \forall\, 1 \leq i \leq N. \tag{5}$$

In this work, we first propose a heuristic dynamic time and wavelength allocation algorithm (Algorithm 1) to allocate wavelength and time slots for upstream transmission in multi-tenant PONs. The dynamic algorithm maintains a few key data structures. A flow SLA breach table tracks how much each traffic flow has breached its SLA in percentage (i.e., going above the non-compliance rate); this is important for minimizing SLA breach, as the algorithm prioritizes scheduling for the flows that have a higher breach of their SLA. A channel freetime table maintains and updates the earliest free time of each channel over time, i.e., when the channel can be reallocated to a different Bmap allocation. We also keep track of the current channel tuned on each transceiver in the ONUs (we consider only one transceiver per ONU). With reference to pseudocode 1, the main procedure **DynamicTwdm** (lines 38–44) is explained as follows. Every time it receives the input Bmaps from the various VNOs, it first calculates the allocation maxtime using the procedure **CalculateMaxtTime** (lines 4–7) in line 41. The allocation maxtime is the latest time an allocation can be scheduled within its latency target [Eq. (1)]. Then, it merges the Bmaps from the VNOs and sorts them based on the following comparison: first, in the descending order of the Bmap allocation flow SLA breach; next, in the ascending order of the allocation maxtimes; and finally, in the ascending order of allocation sizes (line 42). This is explained by the procedure **SortBmaps** (lines 8–19). Then, the procedure **AssignResource** (lines 25–34) is called, which allocates the wavelength and the time slot within that wavelength to each of the sorted Bmap allocations (line 43). In the **AssignResource** procedure, we first find the earliest available wavelength using the sub-procedure **FindMinIndex** (lines 20–24). Then, we check if no channel has been allocated to the ONU or if switching channels is faster than waiting for the currently tuned channel to be free for the transmission of the next Bmap allocation. If this is the case, we allocate the earliest free channel for transmission; else, we stay tuned on the same channel for further transmission (lines 29–32). Then, we update the channel freetime table and OnuTunedChannelMap after the channel assignment to the allocation (line 33) and finally return the corresponding channelwise Bmaps for transmission accordingly (line 34). Finally, we update the SLA breach table using the procedure **UpdateSlaBreachTable** (lines 35–37) in line 44. In this procedure, we calculate the scheduled time for each of the Bmap allocations and update the SLA breach of each VNO flow based on how many allocations of the flow are scheduled beyond the respective allocation maxtimes.

We also present another version, called the static wavelength allocation algorithm (Algorithm 2), where we restrict channel switching in ONUs such that once a channel is allocated to an ONU, the ONU keeps transmitting in the same

Algorithm 1. Dynamic Time and Wavelength Allocation Algorithm

```
1:  Inputs: bmaps, nChannels, channelTuningTime, slaTable
2:  Outputs: mergedBmap
3:  State Tables: slaBreachTable, channelFreeTimeTable,
     onuFreeTimeTable, onuTunedChannelMap
4:  procedure CalculateMaxTime(bmaps)
5:     for bmap in bmaps do
6:        for bmapElem in bmap do
7:           bmapElem.maxTime ← bmapElem.startTime +
     slaTable[bmapElem.vnoId].maxLatency
8:  procedure SortBmaps(bmaps, slaBreachTable)
9:     bmapsMerge ← Flatten(bmaps)
10:    function Compare(elem1, elem2)
11:       slaBreach1 ← slaBreachTable[elem1.vnoId]
12:       slaBreach2 ← slaBreachTable[elem2.vnoId]
13:       if slaBreach1 ≠ slaBreach2 then
14:          return slaBreach1 < slaBreach2
15:       else if elem1.maxTime ≠ elem2.maxTime then
16:          return elem1.maxTime < elem2.maxTime
17:       else
18:          return elem1.size < elem2.size
19:    return MergeSort(bmapsMerge, Compare)
20: procedure FindMinIndex(arr, A)
21:    MinVal ← Min(arr)
22:    MinIndices ← Indices of MinVal in arr
23:    BestIndex ← ArgMin(A[MinIndices])
24:    return BestIndex
25: procedure AssignResource(bmapsSorted,
    channelFreeTimeTable, onuFreeTimeTable,
    onuTunedChannelMap, nChannels, channelTuningTime)
26:    Initialize ChannelAllocCtr and BmapsChannel
27:    for elem in bmapsSorted do
28:       Find EarliestFreeChannel using FindMinIndex
    (ChannelFreeTimeTable, ChannelAllocCtr)
29:       SchedTimeSameChannel ← channelFreeTimeTable
    [current channel]
30:       SchedTimeDiffChannel ← Max(EarliestFreeChannel,
    (channelTuningTime + channelFreeTimeTable
    [current channel]))
31:       switching_faster ← True if SchedTimeDiffChannel <
    SchedTimeSameChannel else False
32:       if elem's current channel unavailable or switching_faster
            then
33:          Assign EarliestFreeChannel
34:       else
35:          Assign current channel
36:       Update onuTunedChannelMap, ChannelFreeTimes, and
    ChannelAllocCtr
37:       return BmapsChannel
38: procedure UpdateSlaBreachTable(BmapsChannel,
    slaBreachTable, numAllocsVno)
39:       for bmapChannel in BmapsChannel do
40:          Update schedTime and SLA breaches for each element
41: procedure DynamicTWDM(bmaps, slaTable, nChannels,
    channelTuningTime)
42:       Initialize state tables
43:       while true do
44:          CalculateMaxTime(bmaps)
45:          BmapsSorted ← SortBmaps(bmaps, slaBreachTable)
                                        (Table continued)
46:          BmapsChannel ← AssignResource(BmapsSorted,
    ChannelFreeTimeTable, onuTunedChannelMap,
    nChannels, channelTuningTime)
47:          UpdateSlaBreachTable(BmapsChannel,
    SlaBreachTable, numAllocsVno)
```

Algorithm 2. Static Wavelength Upstream Scheduling Algorithm

```
1:  Inputs: bmaps, nChannels, slaTable
2:  Outputs: mergedBmap
3:  State Tables: slaBreachTable, onuTunedChannelMap
4:  procedure CalculateMaxTime(bmaps)
5:     for bmap in bmaps do
6:        for bmapElem in bmap do
7:           bmapElem.maxTime ← bmapElem.startTime +
     slaTable[bmapElem.vnoId].maxLatency
8:  procedure SortBmaps(bmaps, nChannels,
     onuTunedChannelMap, slaBreachTable)
9:     Initialize BmapsChannel
10:   for i ← 0 to nChannels − 1 do
11:       for elem ∈ BmapsChannel[i] do
12:          onuId ← elem.onuId
13:          channel ← onuTunedChannelMap[onuId]
14:          Append elem to BmapsChannel[channel]
15:    function Compare(elem1, elem2)
16:          slaBreach1 ← slaBreachTable[elem1.vnoId]
17:          slaBreach2 ← slaBreachTable[elem2.vnoId]
18:          if slaBreach1 ≠ slaBreach2 then
19:             return slaBreach1 < slaBreach2
20:          else if elem1.maxTime ≠ elem2.maxTime then
21:             return elem1.maxTime < elem2.maxTime
22:          else
23:             return elem1.size < elem2.size
24:    for i ← 0 to nChannels − 1 do
25:       BmapsChannel[i]←
       MergeSort(BmapsChannel[i], Compare)
26:    return BmapsChannel
27: procedure UpdateSlaBreachTable(BmapsChannel,
    slaBreachTable, numAllocsVno)
28:    for bmapChannel in BmapsChannel do
29:          Update schedTime and SLA breaches for each element
30: procedure StaticTWDM(bmaps, slaTable, nChannels)
31:    Initialize state tables
32:    while true do
33:          CalculateMaxTime(bmaps)
34:          BmapsChannel ← SortBmaps(bmaps, nChannels,
    onuTunedChannelMap, slaBreachTable)
35:          UpdateSlaBreachTable(BmapsChannel,
    slaBreachTable, numAllocsVno)
```

channel. This can be considered the case with a channel tuning time higher than the PON frame duration (i.e., a slower Class 2 device and Class 3 devices). With reference to the pseudocode (Algorithm 2), such as in the DTWA algorithm, we first calculate the allocation maxtime using the procedure **CalculateMaxtTime** (lines 4–7) in line 33. Then we call the procedure **SortBmaps** (lines 8–26) in line 34. In this procedure, we first separate the Bmap allocations according to the different channels on which they are meant to be transmitted (lines 10–14). Then, we sort the list of Bmap allocations for the

separate channels based on the same comparator function as in the DTWA algorithm (lines 15–25). Finally, we update the SLA breach table in line 35.

We illustrate the difference between the DTWA and SWA algorithms in terms of how we allocate wavelength to the allocations in Fig. 2. Here, we consider three VNOs and a total of five ONUs (we select a small number to simplify the explanation, but this can be extended to larger numbers of ONUs). Each ONU is associated with one of the VNOs as specified by the ONU–VNO table. In this example, the number of transmission channels is two and the channel tuning time considered for all the ONU transceivers is 10 μs. The inputs to the algorithm are three Bmaps with the allocations in each Bmap represented by the same color. For simplicity, here we consider a total of four allocations, but it can be any number of allocations that can fit within a specific time window depending on the time window size and the network line capacity. Each allocation contains information such as the SLA Id, ONU Id, start time, and the grant size of the allocation. We have considered both the start time and grant size to be in μs. In our example, we just show the allocation information for the first allocation in $Bmap_3$ as we will explain only one iteration of allocation conflict resolution and wavelength allocation. Both algorithms maintain two state tables, namely, the SLA breach table, which contains flow-level SLA breach information so far for all the VNOs and an ONU channel table to map the currently tuned channel on ONU transceivers. DTWA also maintains two additional tables, namely, a channel freetime table, which tracks the earliest time when the wavelength would be free again to transmit another allocation and an ONU freetime table, which tracks the earliest time when the ONU would be free again to transmit another allocation. In step 1, for both DTWA and SWA, as explained in code lines 4–7 in both DTWA1 and SWA1, we calculate the maxtimes for all the allocations in each Bmap iteratively (in Fig. 2, we just show the maxtime calculation for the first allocation of $Bmap_3$). Step 2 differs in DTWA and SWA. In the case of DTWA, as explained in code lines 8–19 in Algorithm 1, we append all the Bmaps and sort the appended Bmap, which is explained as follows—for any two allocations in conflict, we first compare the SLA breach of the VNO flows to which the allocations belong respectively. There can be two possible cases as seen from the SLA breach table: case 1, where VNO $flow_3$ has a higher SLA breach than VNO $flow_1$, in which case, the winning allocation will be that of the one belonging to VNO_3, and case 2, where both VNO $flow_1$ and VNO $flow_2$ have the same breach; then we compare the maxtimes of both the allocations that we calculated in step 1, and the winning allocation would be the one with the earlier maxtime. The output of step 2 of DTWA is a sorted merged Bmap. For SWA, in step 2, we separate the allocations into separate channel Bmaps using the ONU channel table (code lines 9–14 in Algorithm 2). All the allocations that will be transmitted in a particular channel are appended together. Hence, the output in this case is a single Bmap for each channel (in our case, it is 2). Step 3 in DTWA is wavelength allocation to each of the sorted Bmap allocations (code lines 25 to 34). There can be two cases to consider—transmit the next allocation at the same wavelength as the ONU is already tuned to or transmit at the earliest available wavelength. In Fig. 2, we have explained the cases illustratively for the first allocation of $Bmap_3$. The allocation belongs to ONU_5, which is already tuned to channel 2. In case 2, the allocation will be scheduled when channel 2 is free for transmission again after 13 μs. But in case 1, if we choose the earliest available wavelength (channel 1 as seen from the channel freetime

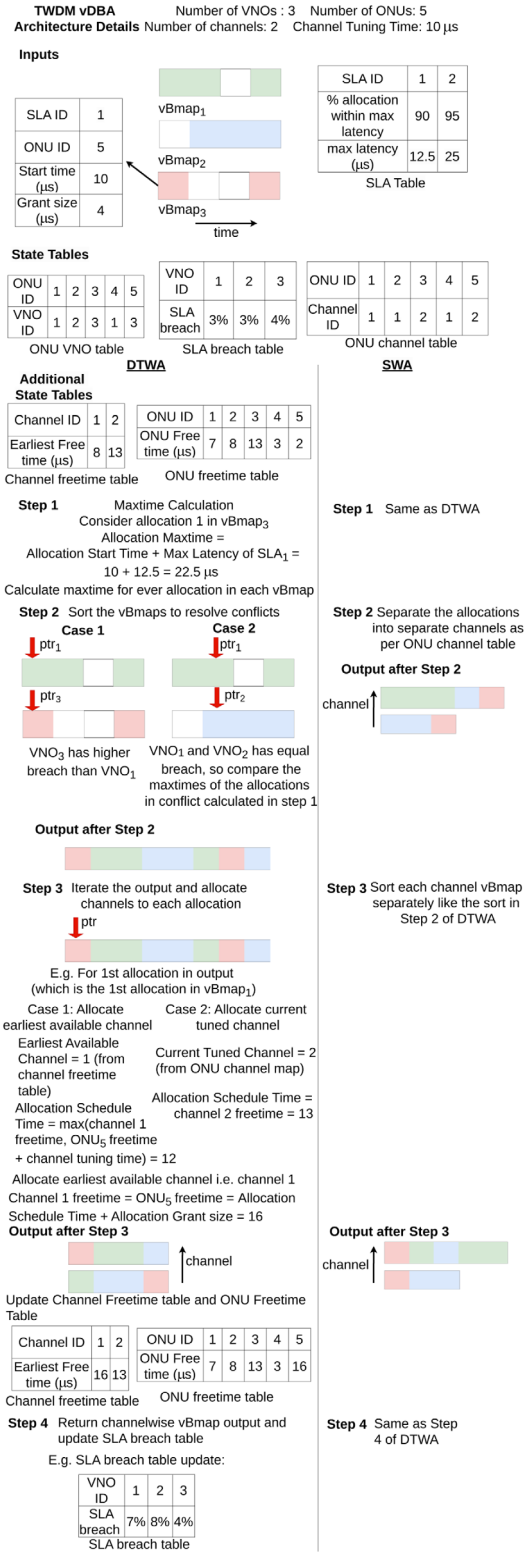

Fig. 2. DTWA versus SWA merging algorithms.

table), the allocation scheduled time will be the maximum of channel 1 freetime and the sum of ONU_5 freetime and channel tuning time, which is 12 μs. Hence, it is always optimal to switch to wavelength 1 for ONU_5 to transmit this allocation. Then, we update the ONU channel table with the new channel (channel 1) just allocated to this ONU, the ONU freetime table and the channel freetime table, which is the sum of the allocation schedule time and the allocation grant size (16 μs). The output after this step in DTWA is the channelwise Bmap. In step 3 of SWA, we sort each channelwise Bmap using the sort function that we described in step 2 of DTWA (code lines 15–26 in Algorithm 2). In step 4 of both DTWA and SWA, we calculate the new SLA breach for each of the VNO flows and update the SLA breach table (code lines 35–37 in Algorithm 1 and code lines 27–29 in Algorithm 2).

3. SIMULATION SETUP FOR OUR PROPOSED ALGORITHM

We carried out a network traffic simulation to evaluate the performance of our merging algorithms: Algorithm 1 and Algorithm 2. The network simulation experiment is shown in Fig. 3. In order to set up the simulation environment, we generate input Bmaps from different VNOs. The allocation load on the shared PON was considered for 20%, 50%, and 80% of the total upstream capacity (i.e., 200 Gb/s across all channels considered). For each of these allocation loads, we then varied the percentage allocated to SLA-driven flows from 10% to 100% of the total load (the remaining part is allocated to best-effort flows). We consider five VNOs [each one generating a virtual Bmap every frame (125 μs)]. We consider a total of 64 ONUs, reflecting the typical split ratio used in both current and emerging PON deployments. These ONUs are evenly distributed among the 5 VNOs, such that 4 VNOs are assigned 13 ONUs each, and 1 VNO is assigned 12 ONUs. To avoid any bias in the VNO-to-ONU assignment, the mapping is randomized across simulation runs. For our experiment, two types of SLAs have been chosen: one requiring 90% compliance with a latency target of 12.5 μs and one requiring 95% compliance with a latency target of 25 μs. This type of QoS cannot be accomplished with classical priority because the stricter requirements for compliance and latency are alternated between the two SLAs. Each bandwidth map has a set of allocations, and the burst sizes were generated from a uniform distribution with the average burst size set at 4% of the total frame size (the same ONU is also allowed to provide multiple bursts per frame). An empty time slot of 0.21 μs is introduced between allocations to account for guard time between upstream transmissions. The minimum and maximum burst sizes were fixed such that the guard times account for 25% and 3% of the burst size, respectively.

The traffic arrival times were generated using four different probability distributions—uniform, Poisson, Zipf–Mendelbrot, and self-similar distributions. Note that we have chosen different distributions for generating the allocation sizes and the allocation arrival times. This is because the allocation sizes depend on the type of user applications, but the arrival times of the allocations depend on the network load, user behavior, and the nature of data transmission. By evenly

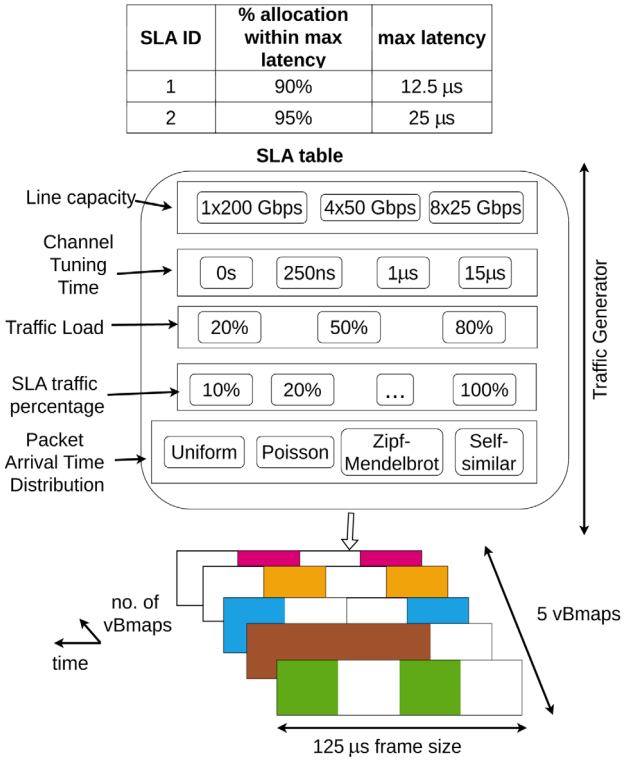

Fig. 3. Simulation experiment.

distributing the allocation sizes within a defined range, we can evaluate how well our merging algorithm performs when handling packets of various sizes. We used the Pareto distribution, which characterizes the self-similar distribution. The various distribution parameters are shown in Fig. 4. Note that here we normalize the distributions for the chosen range of arrival times so as to make sure the arrival times outside that range are unlikely. The arrival times of the Bmap allocations are randomly chosen from a range of 0 to 20 allocation units for each of the distributions. One allocation unit is the smallest upstream grant that can be allocated to one ONU, and for our system, we consider a size of 160 bytes or an approximate duration of 0.05 μs. The choice of range is chosen such that the average arrival time (for uniform and Poisson distributions) is 10% of the average allocation size. This is to make sure that the generated bandwidth maps are not sparse in nature and there is a high probability of allocation conflicts among them. For the uniform distribution, the arrival times are randomly but uniformly chosen from the 0–20 allocation units range with an expected arrival rate of 10 allocation units. The expected arrival rate (λ) for the Poisson distribution is chosen as 10 allocation units, the same as the case of uniform distribution. For the Zipf–Mendelbrot distribution, the exponent characterizing the distribution's tail is $s = 2$. For the Pareto distribution, we have chosen the shape parameter $\alpha = 1$. This ensures that both the Zipf–Mendelbrot and Pareto distributions are highly skewed to the right, meaning shorter arrival times are more likely with rare long delays in between. Each ONU has one transceiver, which is tunable on any of the available channels, and constrained by a tuning time [for the case of the DTWA Algorithm (1)], which is a simulation parameter. As mentioned above, the systems under investigation are 8×25, 4×50, and

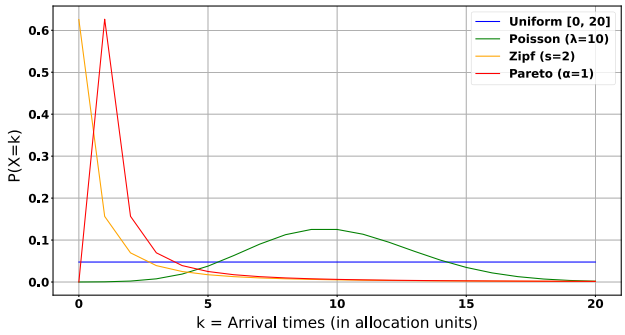

Fig. 4. Normalized traffic arrival rate distributions.

Table 1. Mathematical Notations

Symbol	Description
N	Number of Bmaps from the respective VNOs
M	Number of SLAs
W	Number of channels
F	Frame size in allocation units
SLA_i	SLA ID for the ith VNO
lat_i	Maximum allowed packet level latency for the allocations from the ith VNO
br_{pt}^{i}	Maximum fraction of packets in a flow that can breach packet-level SLA
$\lvert S_i \rvert$	Number of allocations in the ith Bmap
$bw_{i,j}$	jth bandwidth allocation report from the ith Bmap
$req_{i,j}$	Start time of $bw_{i,j}$
$t_{i,j}^{\max}$	Maximum allowed time $bw_{i,j}$ can be delayed without breaching the packet-level SLA
$\mathbb{X}_{i,j,k,l}$	Binary decision variable, $bw_{i,j}$ allotted time slot l of channel k if 1, else 0
$\mathbb{I}$	Boolean truth value function

1×200 Gb/s channels. The tuning times considered span from negligible to 250 ns, 1 µs, and 15 µs. The experiment was run for 1000 time frames, and the average number of SLA breaches was recorded. The experiments were conducted on a Dell Precision 3660 workstation equipped with a 13th Gen Intel Core i9-13900K CPU. The processor has 24 cores (32 threads) with a base clock speed of 3.00 GHz and a maximum turbo frequency of up to 5.80 GHz. The system includes 64 GB of DDR5 RAM (2×32 GB modules) configured at 4400 MT/s. All experiments were executed on Ubuntu 22.04.5 LTS with a Linux kernel 6.8.0-49-generic (see Table 1).

4. RESULTS AND DISCUSSION

A. SLA Performance for Variable Channel Systems and Tuning Times

The results we report in this section demonstrate the system's capability to comply with SLAs as a function of the proportion of flows requiring SLA guarantees. Each figure includes three curves corresponding to PON loads of 20%, 50%, and 80% of total system capacity. Different plot colors represent different channel configurations, specifically 8×25G, 4×50G, and 1×200G transceivers. The performance variation due to different transceiver tuning times is illustrated in Figs. 5–8, and the scheduling behavior is evaluated using Algorithm 1.

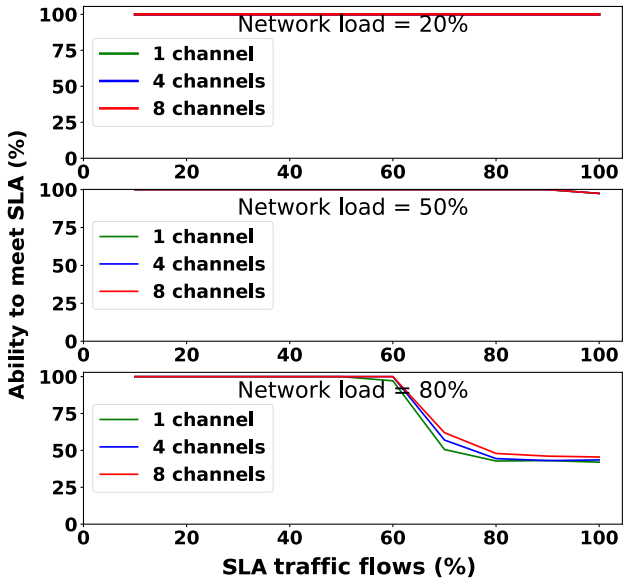

Fig. 5. DTWA loading with a tuning time of 0 s.

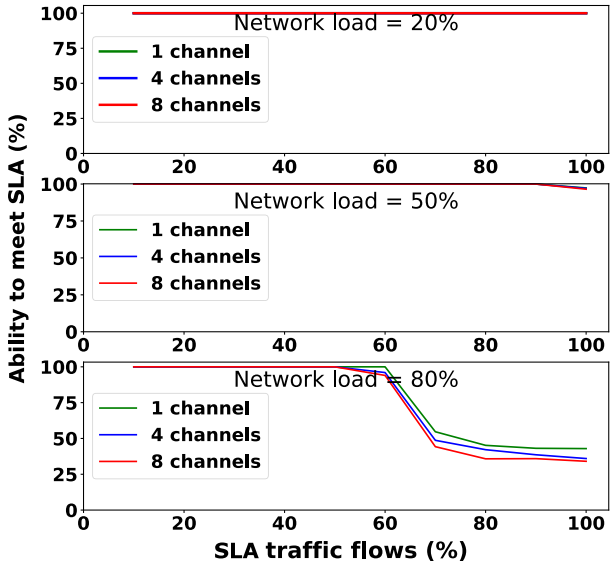

Fig. 6. DTWA loading with a tuning time of 250 ns.

The plot in Fig. 5 reports the ability to meet the SLA when the tuning time is negligible, providing an upper bound on performance as we compare the effect of different tuning times. Under this setting, we observe near-perfect SLA satisfaction for all PON loads up to 50%, with only a minor degradation for SLA traffic fractions exceeding 90%. At 80% PON load, the system begins to show performance degradation: the 8×25G and 4×50G configurations experience SLA violations starting around 60% of SLA traffic, while the 1×200G configuration sees a drop starting at 50%. This confirms that higher aggregate loads increase the contention of slots between different Bmap flows and reduce scheduling flexibility in terms of SLA compliance. Notably, the multi-channel configurations (8×25G and 4×50G) outperform the single-channel system (1×200G), even when tuning time is negligible. This occurs because the PON burst overhead is of the order of a few hundred ns,

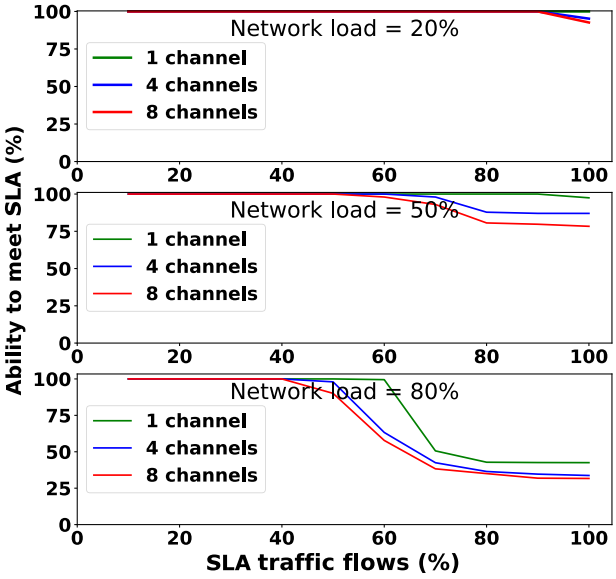

Fig. 7. DTWA loading with a tuning time of 1 μs.

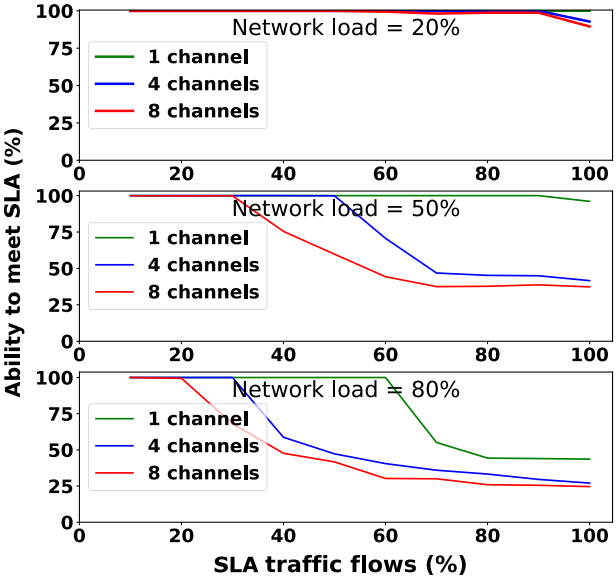

Fig. 8. DTWA loading with a tuning time of 15 μs.

independent of the data rate; thus the drop in throughput is proportionally larger for higher rate channels as they contain a higher number of allocations within the same Bmap time frame.

Figures 6, 7, and 8 analyze the impact of increasing tuning time on SLA compliance. As expected, SLA satisfaction degrades with higher tuning delays. This is because our merging algorithm progressively loses the ability to avoid scheduling delays by tuning to different channels. At a tuning time of 250 ns, the system still performs reasonably well in terms of SLA compliance, but noticeable drops emerge for higher loads and higher SLA traffic ratios. At 1 μs and especially at 15 μs, the performance of the 8×25G and 4×50G systems substantially declines, confirming the strong sensitivity of channel-switching schemes to tuning delays.

Importantly, the 1×200G configuration remains unaffected across all tuning time scenarios. This is because it does not perform any channel tuning—the system operates a single fixed channel, thus incurring no switching overhead. While this setup avoids tuning penalties, it is still limited by higher burst overhead.

The results clearly highlight a fundamental trade-off: while multi-channel configurations provide better statistical multiplexing and finer granularity under ideal tuning conditions, their advantage diminishes or reverses under practical tuning constraints. From a system design perspective, these results suggest that, for architectures with non-negligible tuning times (e.g., ≥250 ns), optimizing channel configuration and tuning policies is critical. Furthermore, these observations emphasize that SLA-aware algorithms must account not only for traffic demand but also for underlying hardware limitations, such as tuning delay, to ensure SLA compliance in high-load scenarios.

Overall, the analysis underscores the necessity of considering physical-layer constraints—especially tuning time—in the design and evaluation of multi-channel PON scheduling algorithms.

B. SLA Performance for Different Traffic Distributions

The impact of different traffic arrival distributions on the average ability to meet SLAs is shown in Fig. 9. To ensure that the results are not biased by specific tuning delays, channel capacities, or network loads, we average the SLA compliance values across all combinations of these parameters considered in the previous experiments. This gives a generalized view of how the traffic distribution alone affects the scheduling algorithm's performance.

The results reveal that the system performs significantly better under uniform and Poisson traffic arrival distributions compared to Zipf–Mendelbrot and self-similar (Pareto) distributions. This can be attributed to the statistical properties of these distributions.

In both the uniform and Poisson distributions, traffic arrivals are more evenly spaced across the allocation window. For the uniform case, each arrival time within the considered range is equally probable, resulting in a low probability of Bmap allocation conflict. In the Poisson distribution, the arrivals are probabilistically centered around a mean (here, 10 allocation units or 0.5 μs), which leads to even better temporal spreading

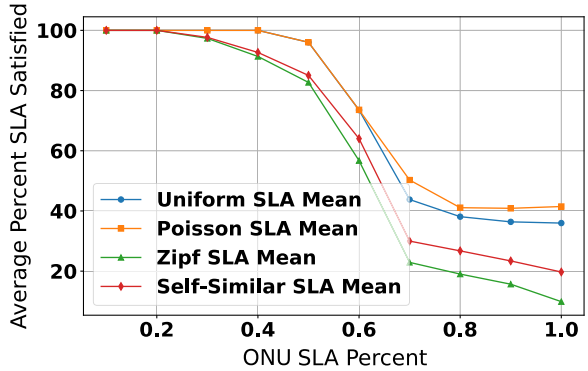

Fig. 9. Average SLA performance of the DTWA algorithm for different traffic distributions.

due to lower variance and higher likelihood of arrivals near the mean. This means that the allocations within each Bmap are temporally dispersed, reducing the chance of allocation collisions and thereby improving SLA compliance.

Moreover, the Poisson distribution slightly outperforms the uniform case. This is because, although both provide fairly symmetrical and moderate-variance arrival profiles, Poisson arrivals tend to cluster mildly around the expected value. This mild clustering minimizes high-variance "gaps" and "bursts" that are more typical in the uniform case, which can lead to variable congestion periods and transient SLA violations.

Conversely, for Zipf–Mendelbrot and self-similar distributions, the probability mass is skewed toward shorter inter-arrival times. These right-skewed distributions lead to temporal clustering of allocation events, which increases the likelihood of contention for transmission windows. In particular, the Zipf–Mendelbrot distribution exhibits a heavier tail than the self-similar distribution, resulting in more extreme short-term bursts. As a result, the performance of DTWA under Zipf–Mendelbrot traffic is significantly degraded due to a high incidence of overlapping requests within a Bmap, which limits the scheduler's flexibility to resolve conflicts within strict SLA deadlines.

These findings emphasize that the effectiveness of the DTWA algorithm is highly dependent on the temporal characteristics of traffic. Distributions that offer more evenly spaced arrivals (with moderate or low variance) enable better scheduling opportunities and lower contention, whereas bursty or highly skewed traffic patterns result in greater conflicts and higher SLA violations.

C. SLA Performance Comparison Between DTWA and SWA

We also examined the SLA performance of the SWA algorithm (Algorithm 2) under a uniform traffic distribution, as shown in Fig. 10. We compare these results with those of DTWA in the case of a 15 μs tuning delay (Fig. 8). We observe that the performance of SWA is comparable to DTWA when the tuning time is high. This is expected, as the advantage of dynamic tuning diminishes with increased tuning delays. At 15 μs, the cost (in terms of idle time and scheduling delay) of switching wavelengths outweighs the potential gain in flexibility. Therefore, assigning a fixed channel per ONU, as in SWA, becomes more effective—or at least equally viable—when tuning incurs high latency penalties.

It is important to note that our experiments assume an equal average load across all ONUs, which helps isolate the impact of short-term allocation strategies. By focusing on short-timescale capacity scheduling, we capture the responsiveness of the algorithms to transient demand variations and contention patterns driven by the arrival distribution, rather than long-term load imbalance.

These results provide valuable insights for the design of bandwidth allocation algorithms in dynamic TWDM-PONs. They suggest that traffic-aware scheduling strategies should account not only for load but also for traffic burstiness and that static channel allocation may be a better alternative in scenarios with hardware-imposed tuning limitations.

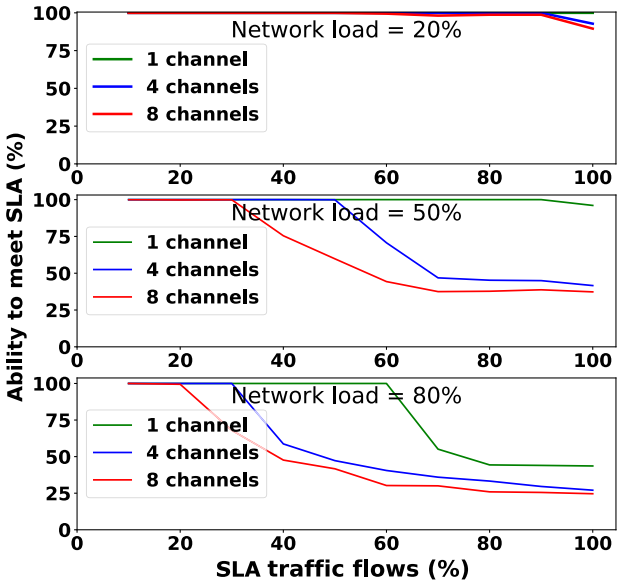

Fig. 10. SLA performance of the SWA algorithm.

D. SLA Performance Comparison Between DTWA and CPLEX

Finally, we compare our DTWA algorithm against the optimal MIP solution provided in Section 2 [through Eqs. (2)–(5)], as shown in Fig. 11. We fix the channel tuning time at 250 ns and the network load at 80% of the maximum possible load for a total aggregated line capacity of 200 Gbps and solve the MIP problem and generate an optimal solution using the IBM ILOG CPLEX Optimizer module. Then we average the SLA performance of the results obtained using CPLEX and DTWA across the different channel capacities. The comparison plot indicates that the performance of our DTWA algorithm is close to optimal until the SLA traffic percentage does not exceed 60%, beyond that, we can observe a significant drop in the performance of DTWA from the optimal CPLEX solution. This is because of fundamental differences in decision-making between DTWA and CPLEX. The search space of the Bmap merging problem is one of the many permutations of all the allocation slots from all the Bmaps. To prune the search space, CPLEX uses a combination of state-of-the-art mathematical optimization algorithms, namely, branch and cut [25], presolve and heuristics [26], cutting planes [27], and dynamic search strategies [28], whereas DTWA is a real-time greedy algorithm, which prioritizes resolving contention based on immediate SLA breaches without performing a full search over potential future outcomes. As SLA traffic increases, contention for slots becomes more intense, and DTWA's heuristic becomes less effective at finding the globally optimal allocation.

E. Algorithm Execution Time

In this section, we report the results of the runtime analysis of the two proposed algorithms. Since the merging algorithm needs to operate in real time, it is important to assess its execution time. The plot in Fig. 12 shows the profiled runtimes of the algorithms run for 1000 occurrences on a 200 Gbps aggregated network capacity system. On average, SWA is

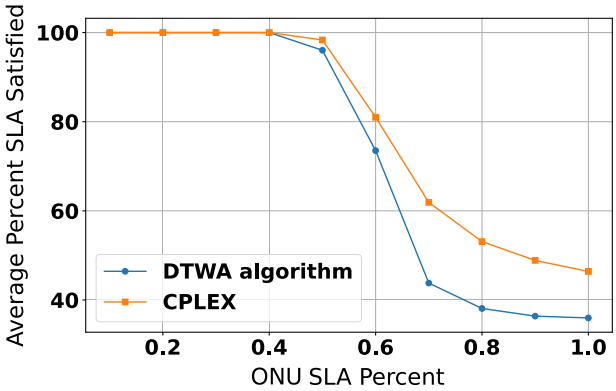

Fig. 11. Average SLA performance of DTWA and CPLEX.

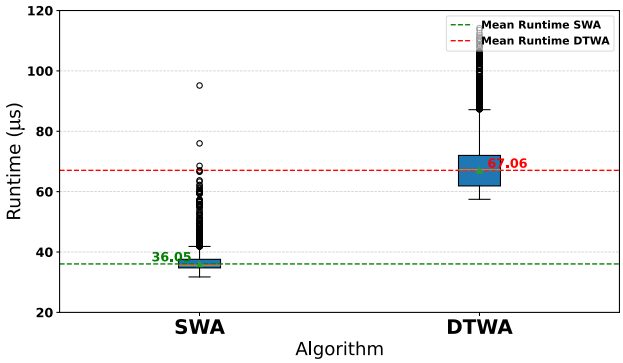

Fig. 12. Boxplot for the runtimes of the algorithms.

1.9 times faster than that of DTWA. The interquartile range (IQR) of SWA is smaller than that of DTWA, which means SWA has a more predictable and consistent runtime. Each step of allocation conflict resolution is accompanied by the selection of an ideal wavelength for transmission in DTWA, whereas in SWA, the choice of wavelength is avoided as the already tuned wavelength is allocated again to the ONU for transmission (Algorithms 1 and 2). The additional system instructions executed for wavelength selection in DTWA lead to higher variance in its runtimes.

We also vary the total line capacity of the system and measure the runtime for both algorithms, as shown in Fig. 13. The runtimes are directly proportional to the line capacity because the total number of allocations (n), combined across all Bmaps, is directly proportional to the amount of data that needs to be scheduled. Both algorithms depend on a resource-intensive merge sort operation, whose time complexity is $\mathcal{O}(n \log n)$, where n represents the number of elements to be sorted. For the more complex DTWA algorithm, the slope is larger than that of SWA. This means that the additional operations introduced while sorting the allocations in DTWA add an extra latency component during the execution of the algorithm.

In addition to the line rate, the value of n also scales with the number of VNOs since each VNO contributes one bandwidth map per frame. A larger number of VNOs increases the total number of allocations to be processed. Furthermore, the number of allocations within each bandwidth map generally increases with the number of ONUs per VNO. However, this increase is not strictly linear, as individual ONUs can issue

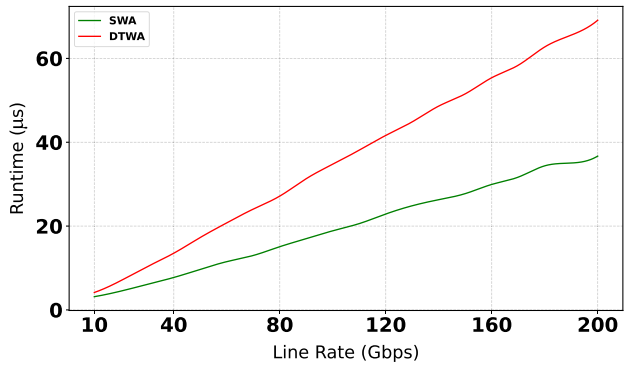

Fig. 13. Runtimes for different line capacities.

multiple allocation requests within the same 125 µs frame under high traffic load. This introduces additional variability and burstiness in the traffic, making the merging process more computationally demanding. As a result, increases in the number of VNOs and ONUs also contribute to the growth of n, and the $\mathcal{O}(n \log n)$ complexity can become a performance bottleneck under such high-load or bursty conditions.

To further support our algorithm's efficiency, we compared DTWA to an optimal solution obtained using the IBM ILOG CPLEX Optimizer. For this comparison, we fixed the tuning time at 250 ns and the network load at 80%, evaluated over all 3 channel capacities, 10 different SLA traffic percentages (from 10% to 100%), and 1000 iterations per configuration, as described in Section 4.D. The total execution time for all CPLEX simulations was 55,520 s. On average, computing a single merged bandwidth map using CPLEX takes approximately 1.85 s, which is several orders of magnitude slower than the DTWA algorithm that executes within a few microseconds.

Overall, we can see that a software-based implementation might be feasible (i.e., an additional latency of less than 10 µs) for rates around 50 Gb/s in a single-wavelength static system, while the dynamic TWDM case would likely require hardware acceleration—especially considering that practical systems may operate with multi-channel configurations at 100 Gb/s and beyond.

5. CHALLENGES AND FUTURE WORK

A. Mitigating SLA Variance Across Traffic Patterns in PON

As discussed in Section 5.B, we observed that SLA compliance was influenced by the nature of traffic distributions in the network. While our proposed merging algorithm achieved high SLA compliance under uniform traffic such as the uniform and Poisson distributions, its performance decreased under bursty or self-similar patterns, such as those modeled by the Pareto or Zipf–Mendelbrot distributions.

Recent studies have underscored the inherent challenges in accurately predicting network traffic at fine timescales. These challenges arise due to the non-stationarity, high variability, and burstiness of traffic in modern broadband and access networks. For instance, bursty traffic often exhibits long-range dependence and heavy-tailed behavior, making fine-grained forecasts of packet arrival rates practically infeasible [29]. Even

deep learning models such as LSTM and GRU fail to generalize under non-repeating traffic patterns and tend to regress to mean predictions [30].

While precise packet-level prediction remains an open challenge, it is considerably more feasible to perform high-level classification of traffic patterns. Classification of ONUs into categories such as *uniform* and *bursty* has been shown to be relatively stable over time and more robust to noise [31,32]. Such classifications can be derived from historical traffic statistics, inter-arrival time distributions, or bandwidth request variability.

To mitigate the SLA performance variance, we propose two potential solutions based on traffic pattern classification:

1. *Distribution-aware DTWA:* by classifying ONUs as *bursty* or *uniform*, we can allocate dedicated transmission windows for bursty ONUs using our DTWA algorithm. This could reduce contention during bandwidth allocation, particularly under high-load or bursty scenarios, and lead to improved SLA compliance.
2. *Map-level bandwidth profile classification:* alternatively, we can perform online classification of ONUs requiring high bandwidth based directly on bandwidth map inputs. This bypasses traffic distribution inference errors but requires additional bandwidth map traversal and processing time. Despite the higher runtime, it offers a more accurate and adaptive bandwidth map merging strategy for real-time deployments.

These classification-based adjustments serve as practical enhancements to our merging algorithm and offer a promising direction for future work. They strike a balance between runtime feasibility and improved performance robustness under diverse traffic conditions.

B. O-RAN Fronthaul Support With PON Integration

We mentioned in Section 1 that vPONs, due to their cost-effectiveness and high scalability, have been considered to support O-RAN fronthaul deployments. We aim to integrate our bandwidth map merging algorithm within such a vPON architecture supporting O-RAN fronthaul. Specifically, we will evaluate the performance of our merging algorithm under latency-sensitive traffic conditions representative of 5G URLLC applications. The goal is to assess how our merging mechanism interacts with cooperative scheduling protocols and whether it can maintain SLA guarantees while supporting real-time mobile traffic over PON.

6. CONCLUSION

In this work, we present a real-time heuristic stateful algorithm for upstream scheduling in a TWDM multi-tenant PON, capable of satisfying SLAs. We show the role that ONU tuning time plays in terms of latency performance in the trade-off between higher capacity single-channel and lower capacity multi-channel systems. Our current results show that tuning times between 250 ns and 1 µs would be required to maintain satisfactory performance in multi-tenant, multi-channel PON systems.

We show that the performance of the algorithm in terms of SLA compliancy is highly affected by the distribution patterns of the upstream network traffic from the ONUs. We show that the runtime of the algorithm is directly proportional to the available network bandwidth.

Our findings also highlight that, although the DTWA algorithm performs well under typical traffic conditions, its SLA compliance drops under bursty or self-similar traffic patterns. We proposed a classification-driven approach to mitigate this variance by identifying bursty ONUs and assigning them transmission windows more appropriately. This not only helps maintain fairness but also reduces resource contention and improves compliance in highly dynamic scenarios.

While the DTWA algorithm demonstrates sub-10 µs runtime performance in software simulation for rates up to 50 Gbps, our runtime scaling analysis shows that higher data rate systems will likely require hardware acceleration. Future integration with PON testbeds based on Intel's DPDK is planned to evaluate the algorithm under continuous real-time operation and hardware-imposed constraints.

Additionally, we plan to evaluate the feasibility of deploying our algorithm in O-RAN fronthaul use cases. Given that URLLC applications have tight latency budgets, we aim to validate whether our merging mechanism can meet end-to-end QoS constraints when deployed within a cooperative DBA framework, as discussed in Section 1 and 3.

Finally, we conclude from our study that our merging algorithm is able to run in 5G and future 6G low-latency services and applications with minimal additional latency.

Funding. Taighde Éireann–Research Ireland (12/RC/2276_p2, 18/RI/5721, 13/RC/2077_p2).

REFERENCES

1. I. Parvez, A. Rahmati, I. Guvenc, *et al.*, "A survey on low latency towards 5G: RAN, core network and caching solutions," IEEE Commun. Surv. Tutorials **20**, 3098–3130 (2018).
2. M. Ruffini, D. B. Payne, and L. Doyle, "Protection strategies for long-reach PON," in *36th European Conference and Exhibition on Optical Communication* (2010).
3. "40-gigabit-capable passive optical networks 2 (NG-PON2): physical media dependent (PMD) layer specification," ITU-T Recommendation G.989.2 (2020).
4. "50-gigabit-capable passive optical networks (50G-PON): physical media dependent (PMD) layer specification," ITU-T Recommendation G.9804.3 (2021).
5. G. Simon, P. Chanclou, M. Wang, *et al.*, "MEC and fixed access networks synergies," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2021), paper Tu1A.3.
6. A. Otaka, "Flexible access system architecture: FASA," NTT Tech. Rev. **15**, 28–34 (2017).
7. A. Elrasad, N. Afraz, and M. Ruffini, "Virtual dynamic bandwidth allocation enabling true PON multi-tenancy," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2017), paper M3I.3.
8. S. Mondal and M. Ruffini, "Optical front/mid-haul with open access-edge server deployment framework for sliced O-RAN," IEEE Trans. Netw. Serv. Manag. **19**, 3202–3219 (2022).
9. T. Tashiro, S. Kuwano, J. Terada, *et al.*, "A novel DBA scheme for TDM-PON based mobile fronthaul," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2014), paper Tu3F.3.
10. J. Maes, S. Bidkar, M. Straub, *et al.*, "Efficient transport of eCPRI fronthaul over PON," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2023), paper W4F.5.

11. G. Kramer, B. Mukherjee, and G. Pesavento, "Interleaved polling with adaptive cycle time (IPACT): protocol design and performance analysis" (2001).
12. J. A. Arokkiam, K. N. Brown, and C. J. Sreenan, "Optimised QoS-aware DBA mechanisms in XG-PON for upstream traffic in LTE backhaul," in *IEEE 4th International Conference on Future Internet of Things and Cloud Workshops (FiCloudW)* (2016), pp. 361–368.
13. L. Yang, Q. Zhang, Z. Huang, *et al.*, "Dynamic bandwidth allocation (DBA) algorithm for passive optical networks," in *30th International Telecommunication Networks and Applications Conference (ITNAC)* (2020).
14. J. Li, X. Shen, L. Chen, *et al.*, "Delay-aware bandwidth slicing for service migration in mobile backhaul networks," J. Opt. Commun. Netw. **11**, B1–B9 (2018).
15. N. Hanaya, Y. Nakayama, M. Yoshino, *et al.*, "Remotely controlled XG-PON DBA with linear prediction for flexible access system architecture," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2018), paper Tu3L.1.
16. M. Ruffini, A. Ahmad, S. Zeb, *et al.*, "Virtual DBA: virtualizing passive optical networks to enable multi-service operation in true multi-tenant environments," J. Opt. Commun. Netw. **12**, B63–B73 (2020).
17. C. Li, W. Guo, W. Wang, *et al.*, "PON bandwidth resource sharing schemes in a multi-operator scenario," in *International Conference on Computing, Networking and Communications (ICNC)* (2017), pp. 397–401.
18. C. Li, W. Guo, W. Wang, *et al.*, "Bandwidth resource sharing on the XG-PON transmission convergence layer in a multi-operator scenario," J. Opt. Commun. Netw. **8**, 835–843 (2016).
19. A. Ahmad, A. Ahmed, A. Al-Dweik, *et al.*, "Merging engine implementation for intra-frame sharing in multi-tenant virtual passive optical networks," J. Opt. Commun. Netw. **15**, 209–218 (2023).
20. K. Mohammadani, R. Butt, K. Ali, *et al.*, "Load adaptive merging algorithm for multi-tenant PON environments," Opt. Switching Netw. **47**, 100712 (2022).
21. F. Slyne, S. Zeb, and M. Ruffini, "Stateful DBA hypervisor supporting SLAs with low latency & high availability in shared PON," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2021), paper W6A.48.
22. A. Ganguli, F. Slyne, and M. Ruffini, "Real-time, low latency virtual DBA hypervisor for SLA-compliant multi-service operations over shared passive optical networks," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2023), paper Tu3F.2.
23. A. Ganguli and M. Ruffini, "Low-latency upstream scheduling in multi-tenant, SLA compliant TWDM PON," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2024), paper Tu3I.3.
24. L. Zhang, Y. Luo, B. Gao, *et al.*, "Channel bonding design for 100 gb/s PON based on FEC codeword alignment," in *Optical Fiber Communication Conference (OFC)* (Optica Publishing Group, 2017), paper Th2A.26.
25. IBM Corporation, IBM ILOG CPLEX Optimization Studio: Branch-and-Cut (2023).
26. IBM Corporation, IBM ILOG CPLEX Optimization Studio: MIP Heuristics (2023).
27. IBM Corporation, IBM ILOG CPLEX Optimization Studio: Cutting-Planes (2023).
28. IBM Corporation, IBM ILOG CPLEX Optimization Studio: Dynamic-Search (2023).
29. P. Jahnke, E. Stapf, J. Mieseler, *et al.*, "Towards fine grained network flow prediction," arXiv (2018).
30. P. Jahnke, "Network traffic prediction with neural networks," (2019–2020).
31. N. E. Frigui, T. Lemlouma, S. Gosselin, *et al.*, "Optimization of the upstream bandwidth allocation in passive optical networks using internet users' behavior forecast," in *International Conference on Optical Network Design and Modeling (ONDM)* (2018), pp. 59–64.
32. H. Oudah, B. Ghita, T. Bakhshi, *et al.*, "Using burstiness for network applications classification," J. Comput. Netw. Commun. **2019**, 5758437 (2019).